Compiler Design Project Report Syntax Error Detection in C Language using Python

Submitted By
Hetvi Sheth [23001099]
Parisha Parmar [23001101]
Priya Bhatt [23001125]

Department of Computer Science and Engineering
Navrachana University, Vadodara

Abstract

This project focuses on the design and implementation of a Python-based C Syntax Checker capable of identifying various syntax and semantic errors in C programming language programs. The project combines compiler design concepts with data structures such as Stack, Symbol Table, Parse Tree, FSM, Linked List, and Token Stream to validate program structure and generate meaningful error messages.

Introduction

Compiler Design plays an important role in translating high-level programming languages into machine-readable code. One of the major responsibilities of a compiler is syntax analysis and error detection. This project aims to build a mini syntax checker that can automatically identify common errors in C programs.

Objectives

1. To identify common syntax and semantic errors in C language.
2. To study compiler-related data structures and their applications.
3. To implement syntax error detection using Python.
4. To generate meaningful and user-friendly error messages.
5. To test and validate the syntax checker using different invalid C programs.

Technologies Used

- Python Programming Language
- Regular Expressions (Regex)
- Stack Data Structure
- Symbol Table
- Parse Tree Concepts
- Finite State Machine (FSM)
- Token Stream Analysis

Phase 1 – Identification of C Language Errors

In the first phase, different syntax and semantic errors in C language were studied and classified. A total of 50 different syntax and semantic errors were identified during this phase.

Phase 2 – Mapping Errors with Data Structures

Suitable compiler data structures were mapped with corresponding syntax errors for efficient error detection and syntax validation.

Phase 3 – Implementation of Syntax Checker

The actual implementation of the C Syntax Checker was completed using Python. Different modules were developed for bracket checking, semicolon validation, loop syntax checking, function declaration validation, and preprocessor directive validation.

Phase 4 – Testing and Validation

Different incorrect C programs were tested using the developed syntax checker. Outputs generated by the system were compared with expected results to verify correctness and efficiency.

Conclusion

The Compiler Design project successfully implemented a Python-based C Syntax Checker capable of identifying multiple syntax errors in C programs. The project provided practical understanding of compiler techniques, data structures, and syntax analysis mechanisms.

Future Scope

• Automatic syntax correction
• Real-time error detection
• GUI-based syntax checker
• IDE integration
• Advanced semantic analysis
• Code suggestion and auto-completion features

Example Mapping Table

Error Type	Data Structure
Bracket Error	Stack
Variable Declaration Error	Symbol Table
Function Errors	Symbol Table
String Literal Error	FSM + Stack
Control Statement Error	Parse Tree